

Base de conocimiento — Alberto Blasco Ventas

Documento diseñado como **fuentes largas** para un chatbot (retrieval) y para perfilar mi trabajo en **IA Generativa**, **backend con FastAPI/Docker**, **RAG**, **bases de datos vectoriales (Qdrant)** y **sistemas de agentes** (LangGraph, CrewAI, ReAct).

0) Perfil

- **Nombre:** Alberto Blasco Ventas
 - **Fecha de nacimiento:** 30/08/2001 (24 años)
 - **Rol/Especialidad:** Ingeniero Informático | **IA Generativa** y **Backend**
 - **Intereses clave:** Copilots y asistentes basados en RAG, evaluación de LLMs, seguridad/privacidad, MLOps/LLMOps, automatización con Power Platform y visualización con Power BI.
-

1) Proyecto — OpenGenServer (FastAPI + Docker + CI/CD + Hugging Face Spaces)

Resumen corto: Backend **production-ready** con **FastAPI**, totalmente **contenedorizado** con **Docker**, **CI/CD** (GitHub Actions) y **despliegue** como **Space Docker** en Hugging Face. Proporciona endpoints para **autenticación**, **gestión de proyectos/items** y **tareas de IA generativa** (texto y embeddings), con arquitectura modular y preparada para escalar.

1.1 Objetivos de diseño

- **Modularidad y limpieza:** routers/servicios/esquemas separados.
- **Portabilidad:** imagen Docker reproducible, `.dockerignore` cuidado.
- **Operativa real:** CI/CD con test + build + deploy automático a HF Spaces.

- **Seguridad básica:** JWT para autenticación y control de acceso.
- **Observabilidad mínima viable:** health-check, logs estructurados y trazas a futuro.

1.2 Stack técnico

- **Backend:** FastAPI (Python 3.11), Uvicorn.
- **CI/CD:** GitHub Actions ([.github/workflows/ci.yml](#)).
- **Infra/Hosting:** Hugging Face Spaces (modo Docker).
- **Contenedor:** [Dockerfile](#) optimizado, multi-stage opcional.
- **Dependencias:** [requirements.txt](#).

1.3 Estructura de proyecto (carpetas/archivos)

```

.
├── app/
│   ├── main.py          # Punto de entrada FastAPI
│   ├── routers_sql/     # Routers: auth, items, ai, etc.
│   ├── core/            # Seguridad, config, utilidades
│   └── models/          # Modelos de BD (si aplica)
├── .github/workflows/ci.yml # Pipeline GitHub Actions
├── Dockerfile           # Build Docker
├── .dockerignore        # Exclusiones para Docker
├── requirements.txt     # Dependencias
└── README.md            # Documentación

```

1.4 Endpoints (resumen)

Autenticación

- [POST /auth/register](#) — alta de usuario (hash de password, validaciones).
- [POST /auth/login](#) — login, emisión **JWT**.
- [GET /auth/me](#) — información del usuario autenticado.

Items/Proyectos

- [POST /items](#) — crear item (auth requerida).

- `GET /items` — listar items.

IA Generativa

- `POST /ai/generate` — generación de texto.
- `POST /ai/embed` — generación de embeddings.
- `POST /ai/generate/stream` — **SSE** para streaming de texto.

Utilidad

- `GET /health` — health check.
- `GET /docs` — **Swagger UI**.
- `GET /redoc` — **ReDoc**.

1.5 Configuración y ejecuciones

Variables de entorno (`.env` local o Variables en el Space):

- `SECRET_KEY` — firma de JWT.
- `ACCESS_TOKEN_EXPIRE_MINUTES` — minutos de expiración del token (p.e. 60).
- `APP_ENV` — `dev` | `prod`.

Local (sin Docker)

```
uvicorn app.main:app --reload  
# http://127.0.0.1:8000
```

Docker local

```
docker build -t opengenserver .  
docker run -p 7860:7860 opengenserver  
# http://127.0.0.1:7860
```

Despliegue (HF Spaces Docker)

- Push a GitHub ⇒ CI corre tests ⇒ build de imagen ⇒ deploy a Space.
- Acceso: <https://<usuario>-<space>.hf.space/> y documentación en </docs>.

1.6 Seguridad, calidad y observabilidad

- **Seguridad:** JWT, validación de payloads (Pydantic), CORS configurado, [.dockerignore](#) para evitar credenciales en las capas.
- **Calidad:** tests (pytest), tipos (mypy opcional), format (black/ruff opcional), PRs con checks.
- **Observabilidad:** logs estructurados, plan para métricas (Prometheus/Grafana/OTEL), trazas por request, IDs de correlación.

1.7 Extensiones recomendadas (roadmap)

- **Rate limiting** y **API Keys** por cliente.
- **Feature flags** y **versionado de API**.
- **Retrieval (RAG)** con vector DB para endpoints de IA.
- **RBAC** (roles/permisos) y auditoría.
- **Cache** de respuestas generativas (p.ej., Redis) y colas (RQ/Celery).

1.8 Notas de licencia

- Mantener **consistencia** entre el campo [license](#) del README/Space y el archivo de licencia del repo. (Si README indica MIT y metadata Apache-2.0, unificar.)

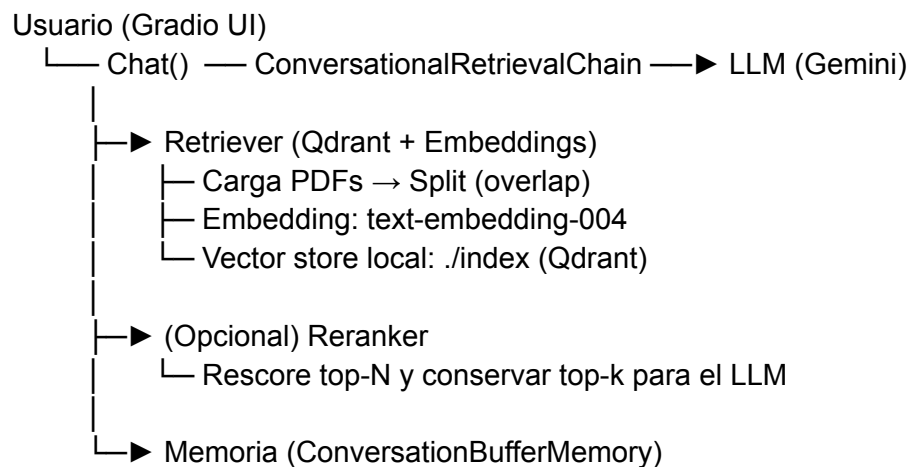
2) Proyecto — Simple PDF RAG Chatbot (Gradio + Gemini + Qdrant)

Resumen corto: Chatbot **RAG** que contesta preguntas sobre **tus PDFs** con **citas y páginas**. Usa **Gemini** como LLM, **text-embedding-004** para embeddings, y **Qdrant** en **modo local (path)** para la búsqueda vectorial. Interfaz en **Gradio**, con **memoria conversacional**, ajustes de **retriever** dinámicos y **IDs de chunk deterministas** para estabilidad de referencias. Incluye **reranking opcional**.

2.1 Características destacadas

- **Privacidad por diseño:** indexación local en `./index` (Qdrant path mode).
- **Citas reproducibles:** IDs deterministas de chunks $\Rightarrow$ estabilidad entre runs.
- **Memoria conversacional:** contexto multi-turn en la cadena.
- **Ajustes en caliente:** `k`, `fetch_k`, `lambda_mult`, estrategia (`mmr/similarity`).
- **Reranking opcional:** reordenación fina del pool recuperado antes del LLM.
- **UI clara** con Gradio: subida de PDFs, controles de recuperación y estado.

2.2 Arquitectura (alto nivel)



2.3 Configuración del Retriever (valores recomendados)

- `vectore_store_k = 5`
- `vectore_store_fetch_k = 30` (aumentar a 50–100 si se necesita más recall)
- `vectore_store_lambda_mult = 0.5` (si se usa **MMR** para diversidad)
- **Heurística mixta** MMR + Rerank: seleccionar un pool con MMR y rerankearlo; o mezclar `ceil(k/2)` MMR + `ceil(k/2)` similarity antes del rerank final.

2.4 Flujo de trabajo

1. Cargar PDFs en `data/`.

2. Particionar con **RecursiveCharacterTextSplitter** (overlap controlado).
3. Generar **embeddings** con `text-embedding-004`.
4. Indexar en **Qdrant** (colección p.ej. `my_collection`).
5. Recuperar candidatos (MMR/similarity) con `fetch_k` alto.
6. **(Opcional) Reranking** para precisión.
7. Pasar `top_k` chunks + query a **Gemini**.
8. Responder con **citas** (archivo + página) y mantener **memoria**.

2.5 Tests y validaciones sugeridas

- **Cross-PDF**: consultas que crucen varios documentos.
- **Sensibilidad a parámetros**: variar `k`, `fetch_k`, `lambda_mult` y medir calidad/latencia.
- **Reranking ON/OFF**: comparar precisión subjetiva y tiempos.
- **Determinismo de citas**: estabilidad de IDs en runs repetidos.
- **Memoria**: resetear y comprobar retención entre turnos.

2.6 Roadmap

- **Híbrido lexical+vectorial** (BM25 + embeddings).
- **Rerankers LLM/cross-encoder calibrados**.
- **Multilingüe y clickable citations** con vista previa PDF.
- **Backend FastAPI y Docker one-command**.

2.7 Requisitos, ejecución y variables

- **Python 3.10+**.
- Dependencias: `langchain`, `langchain-community`, `langchain-google-genai`, `qdrant-client`, `gradio`, `python-dotenv` (+

extras para el reranker si aplica).

- `.env` $\Rightarrow$ `GOOGLE_API_KEY=...`
 - Ejecución: `python app.py` $\Rightarrow$ `http://127.0.0.1:7860`.
-

3) Bases de datos vectoriales y recuperación

- **Qdrant**: base de datos vectorial de alto rendimiento (modo local path o cloud). Soporta filtros por metadatos, búsquedas por similitud, MMR, distancias configurables (p.ej. coseno), índices optimizados.
 - **Buenas prácticas**:
 - Diseñar **esquema de metadatos** (título, autor, página, sección, ruta del fichero) para filtros y trazabilidad.
 - Controlar **chunking**: tamaño y solape afectan recall/precision.
 - **Versionar** colecciones y mantener **migraciones** de índice.
 - Medir **latencia** y **recall@k**; añadir **cache** de consultas frecuentes.
-

4) Sistemas de Agentes y orquestación

- **LangGraph** (ecosistema LangChain): framework para **workflows/agents** con estado, streaming, depuración y despliegue; útil para **multi-agente**, **bucles** controlados y **FSM**.
- **CrewAI**: orquestación **multi-agente** orientada a colaboración (roles, tareas, herramientas, delegación, memoria) con APIs y studio para componer equipos.
- **ReAct (Reason + Act)**: patrón de prompting/arquitectura donde el agente **razona** y **actúa** de forma intercalada (tool use) para tareas complejas; incrementa trazabilidad de razonamiento y capacidad de recuperación/planificación.

4.1 Patrón de trabajo con agentes

1. **Definir objetivos/roles y herramientas** (tooling) de cada agente.
2. **Control de flujo**: gráfico/estado (LangGraph) o crews (CrewAI) con delegación y coordinación.
3. **Memoria y contexto**: buffers, memorias vectoriales, memoria episódica.
4. **Guardrails**: validadores, políticas de seguridad/PII, límites de coste/latencia.
5. **Evaluación**: juegos de prueba (task suites), métricas de éxito por tarea, trazas por paso.

4.2 Casos de uso que he abordado

- **Extracción/Enriquecimiento** sobre documentos (agentes lectores + verificadores + sintetizadores).
 - **RAG con agentes**: separación rol recuperador vs. rol escritor, + un verificador que contrasta citas.
 - **Automatización**: agentes con herramientas (buscadores, APIs internas, bases de conocimiento, Power Automate) que ejecutan acciones y reportan.
-

5) Métricas y evaluación (LLM/RAG/Agentes)

- **Calidad**: factualidad, completitud, coherencia, cobertura de cita, groundedness.
- **Recuperación**: recall@k, MRR, precisión post-rerank, tiempo de indexado.
- **Sistema**: latencia P50/P95, coste por consulta, tasa de errores (timeouts/tool failures).
- **Producto**: adopción, satisfacción (CSAT), ahorro de tiempo/horas, tareas resueltas por semana.

Pipelines de evaluación: conjuntos de consultas «duras», veredictos automáticos (LLM-as-judge con rúbricas) + muestreo humano; dashboards de evolución.

6) Seguridad, privacidad y cumplimiento

- **JWT** para control de acceso; **scopes** por endpoint.
 - **PII**: reglas de redacción/masking; políticas de retención.
 - **Rate limiting**, **quotas** por usuario/cliente, backoff exponencial.
 - **Audit logs y trazabilidad** de prompts/completions/acciones.
 - **Secret management**: variables de entorno, vaults; nunca almacenar llaves en repositorios.
-

7) Operativa/MLOps/LLMOps

- **Versionado** de prompts, retrievers y colecciones.
 - **CI/CD** con pruebas y despliegues canary.
 - **Observabilidad**: logs, métricas (Prometheus), trazas (OTel), paneles (Grafana, Power BI para negocio).
 - **Cost control**: batch, cache, truncación de contexto, compresión de historial, políticas de reintentos.
-

8) FAQs (para el chatbot)

- **¿Puedo cambiar el LLM?** Sí; la arquitectura aísla la llamada al modelo. Ajustar prompts y evaluación.
- **¿Dónde se guardan mis documentos?** En local (`./data`) y el índice vectorial en `./index` (Qdrant path mode), salvo configuración cloud.
- **¿Cómo activo el reranking?** Toggle en UI o flag en código; se aplica tras recuperar el pool.
- **¿Cómo escalo OpenGenServer?** Usar workers Uvicorn/Gunicorn, autoescalado del Space o despliegue en otra infra; añadir caché y colas.
- **¿Cómo protejo endpoints de IA?** JWT + API Keys por cliente, rate limiting, guardrails de contenido.

9) Glosario breve

- **RAG**: Retrieval-Augmented Generation, generación condicionada a contexto recuperado.
- **MMR**: Maximal Marginal Relevance, equilibrio relevancia/diversidad en recuperación.
- **Reranking**: reordenamiento fino del set recuperado con un modelo adicional.
- **ReAct**: Reason+Act; alternancia de razonamiento y acción (uso de herramientas).
- **Qdrant**: base de datos vectorial open-source para similitud de alta dimensión.
- **LangGraph**: framework para workflows/agents con estado en el ecosistema LangChain.
- **CrewAI**: framework para equipos multi-agente con roles/herramientas/memoria.

10) Resumen de mi experiencia (para copiar en perfil)

- Desarrollo de **backends** de IA con **FastAPI** y **Docker**, CI/CD y despliegue en **Hugging Face Spaces**.
- Construcción de **chatbots RAG** con **Gemini**, **Qdrant**, **LangChain/Gradio**; citas por **página** y **memoria**.
- Trabajo con **bases vectoriales** (Qdrant), estrategias **MMR/similarity**, **reranking**, diseño de metadatos.
- Orquestación de **sistemas de agentes** con **LangGraph** y **CrewAI**; aplicación del patrón **ReAct**.
- Enfoque en **seguridad**, **observabilidad**, **evaluación** y **KPIs de producto**.

11) Próximos pasos sugeridos

- Publicar una **demo Docker one-command** del RAG chatbot.

- Añadir **API REST** (FastAPI) al RAG para integración con OpenGenServer.
- Incorporar **evaluación automática** (datasets + LLM-as-judge) y panel de métricas.
- Unificar **licencia** y badges de estado (build, tests, Space live).

Autor: Alberto Blasco Ventas · **Contacto:** albertoblasco55555@gmail.com